02 November 2025 13:24

Sum of digits

Input :) n : 346

output :) 3+4+6 : 13

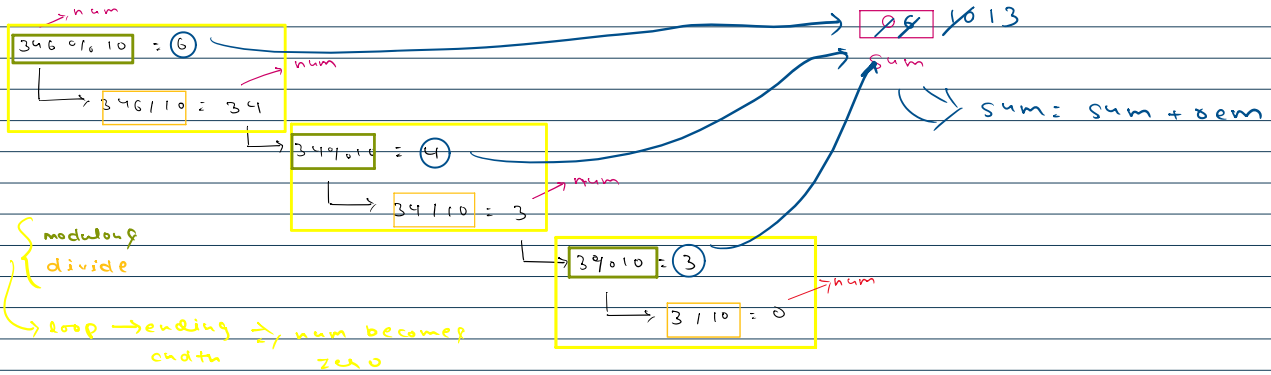
346 : 3x100 + 4x10 + 6x1

346 %. 10 : (6)

10) 346 (34
340

6

How to find the last digit




```
while ( n > 0 ) {
    int rem = num %. 10 ;
    num : num / 10 ;
    Sum : Sum + rem ;
}
```

```
public class Main {
    public static void main(String[] args) {
        int num = 346;
        int sum = 0;

        while(num > 0){
            int rem = num % 10;
            num = num/10;
            sum = sum + rem;
        }
        System.out.println(sum);
    }
}
```

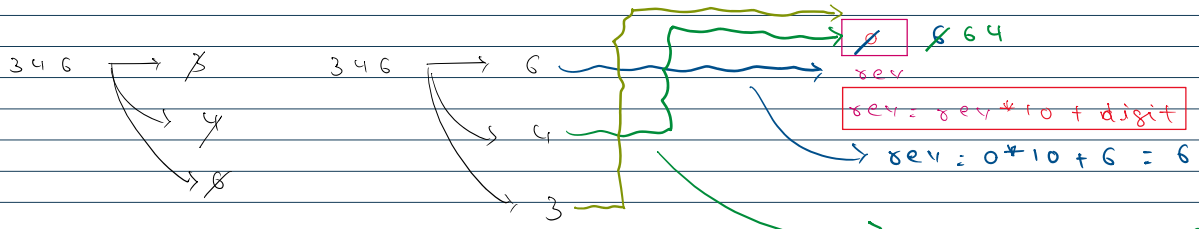
Reverse a number

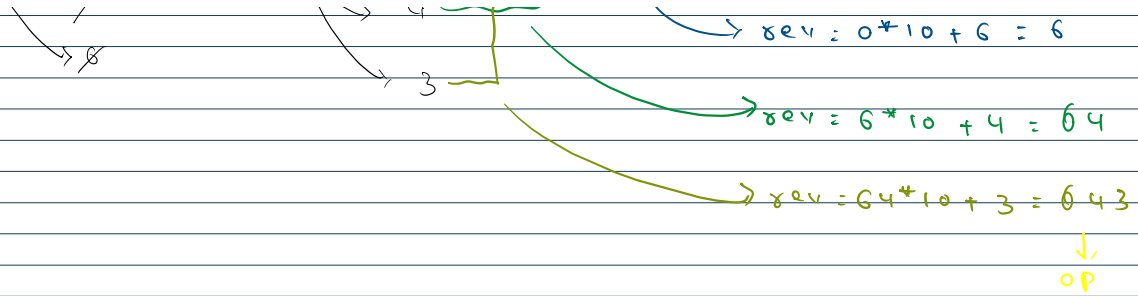
Input :) n : 346 ;

output :) 643

346 : 3*100 + 4*10 + 6*1

643 : 6*100 + 4*10 + 3*1






```
public class Main {
    public static void main(String[] args) {
        int num = 346;
        int rev = 0;

        while(num > 0){
            int rem = num % 10;
            num = num/10;
            rev = rev*10 + rem;
        }

        System.out.println(rev);
    }

}
```

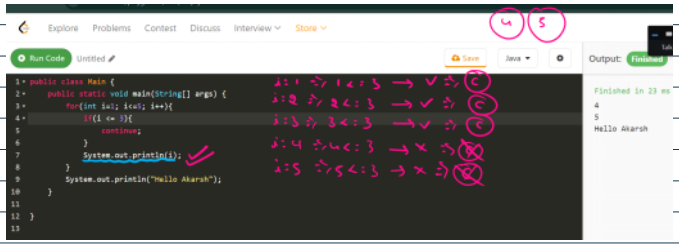
Break statement

exit a loop immediately when a certain condition occurs

```
 public class Main {
     public static void main(String[] args) {
         for(int i=1; i<=5; i++){
             if(i == 3){
                 break;
             }
             System.out.println(i);
         }
         System.out.println("Hello Akarsh");
     }

 }
```

```
public class CheckPrime {
    public static void main(String[] args) {
        int n = 9;
        int count = 0;
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                count++;
                    break;
            }
        }
        if (count >= 1) {
            System.out.println("Not Prime");
        } else {
            System.out.println("Prime");
        }
    }

}
```

Continue statement




```java
public class Main {
    public static void main(String[] args) {
        for(int i=1; i<=5; i++){
            if(i <= 3){
                continue;
            }
            System.out.println(i);
        }
        System.out.println("Hello Akarsh");
    }

}
```

Fibonacci Sequence (Series)

0 1 1 2 3 5 8 11

f(n) = f(n-1) + f(n-2)

first → 0
second → 1

loop

a = 0
b = 1
c = a + b

a = b
b = c

0 1 1 2 3 5 8

a → second last term

b → last term

a b c

a = 0
b = 1
c = a + b = 1

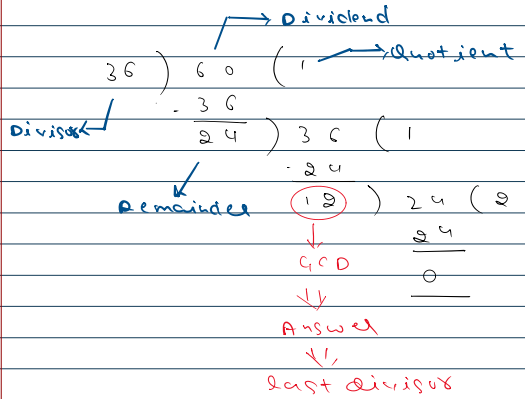
c = a + b = 1 + 1 = 2

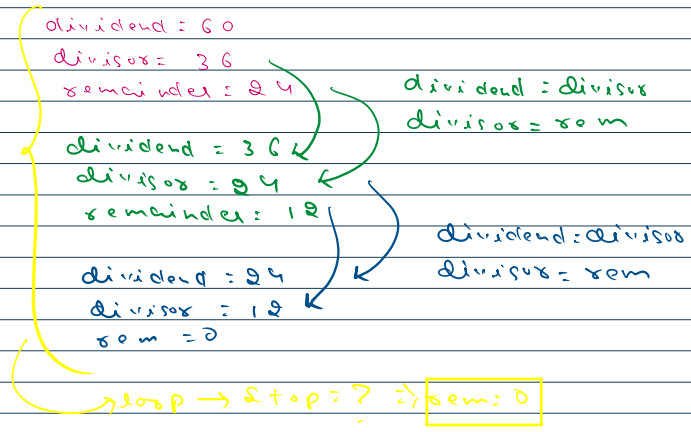
```java
public class Main {
    public static void main(String[] args) {
        int n = 2;

        int a = 0;
        int b = 1;
        int c = 0;
        for(int i=3; i <= n; i++){
            c = a + b;

            a = b;
            b = c;
        }
        System.out.println(c);
```

```
    }
}
```

Greatest Common Divisor (GCD)

GCD(60, 36)






```
public class Main {
    public static void main(String[] args) {
        int dividend = 36;
        int divisor = 60;

        while(dividend % divisor != 0){
            int rem = dividend % divisor;
            dividend = divisor;
            divisor = rem;
        }
        System.out.println(divisor);
    }

}
```

Number System

	Base	Digits used	Example
Decimal	10	0 - 9	456
Binary	2	0 & 1	100, 101, 111
Octal	8	0 - 7	012, 345, 567
Hexadecimal	16	0-9, A-F	A12

$456 = 4*10^2 + 5*10^1 + 6*10^0$

Java => Binary Representation

=> 2's complement representation

Signed bit => leftmost bit

=> 0 => positive

=> 1 => negative

Two's complement

5 = 101

16 : 1000

2	16	0
2	8	0
2	4	0
2	2	0
	1	

→ 1000

2	5	1
2	2	0
	1	

→ 101

Decimal to Binary

One's complement

5 => 00000101

↓ 1's complement

11111010

Addition =>

```
  1
  99
+  1
-----
 100
```

```
  010
+  1
-----
  011
```

```
  11
  111   → 7
+   1   → 1
--------
 1000   → 8
```

1+1 → 10

1+1 → 2

↓

(10)

```
  11
  111   → 7
+  11   → 3
--------
 1010   → 10
```

1+1+1 → 3

↓

11

Two's complement = One's complement + 1

+5 => 00000101

↓ 1's complement (invert all bits)

11111010

+ 1

-5 => 11111011

→ verify => (+5) + (-5) = 0

```
  11111111
+5 => 00000101
-5 => 11111011
---------------
 1 00000000
```

1+1 = 10

1+0 = 1

↓

{carry = 1}

→ extra → discarded bit

Range of data types

Size of data types (bits)

Size of data types

type	size (bits)	signed range	unsigned range
byte	8	-128 to +127	0 to 255
short	16	-32768 to 32767	0 to 65535
int	32	-2^{31} to $2^{31}-1$	0 to $2^{32}-1$
long	64	-2^{63} to $2^{63}-1$	0 to $2^{64}-1$

Range unsigned

$2^7\times0 + 2^6\times0 + 2^5\times0 + 2^4\times0 + 2^3\times0 + 2^2\times0 + 2^1\times0 + 2^0\times0$

0

0 0 0 0 0 0 0 0 => 0

1 1 1 1 1 1 1 1 => 255

$2^7\times1 + 2^6\times1 + 2^5\times1 + 2^4\times1 + 2^3\times1 + 2^2\times1 + 2^1\times1 + 2^0\times1$

128 + 64 + 32 + 16 + 8 + 4 + 2 + 1

255

Range signed

1 0 0 0 0 0 0 0 => $(-2^7)\times1 + 2^6\times0 + 2^5\times0 + 2^4\times0 + 2^3\times0 + 2^2\times0 + 2^1\times0$

=> $(-128)\times1 + 0*8$

=> -128

0 1 1 1 1 1 1 1 => $2^7\times0 + 2^6\times1 + 2^5\times1 + 2^4\times1 + 2^3\times1 + 2^2\times1 + 2^1\times1 + 2^0\times1$

=> 0 + 64 + 32 + 16 + 8 + 4 + 2 + 1

=> 127

Type Conversion

Changing a variable's data type.

1) Implicit conversion (Type promotion)
automatically change

smaller type ⟶ larger type

Promotion order: bit => byte => short => int => long => float => double

2) Explicit conversion (Type conversion)

Lossy Lossless

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        double b = a;
        System.out.println(b);
```

$0 \times 2^{5} + 0$

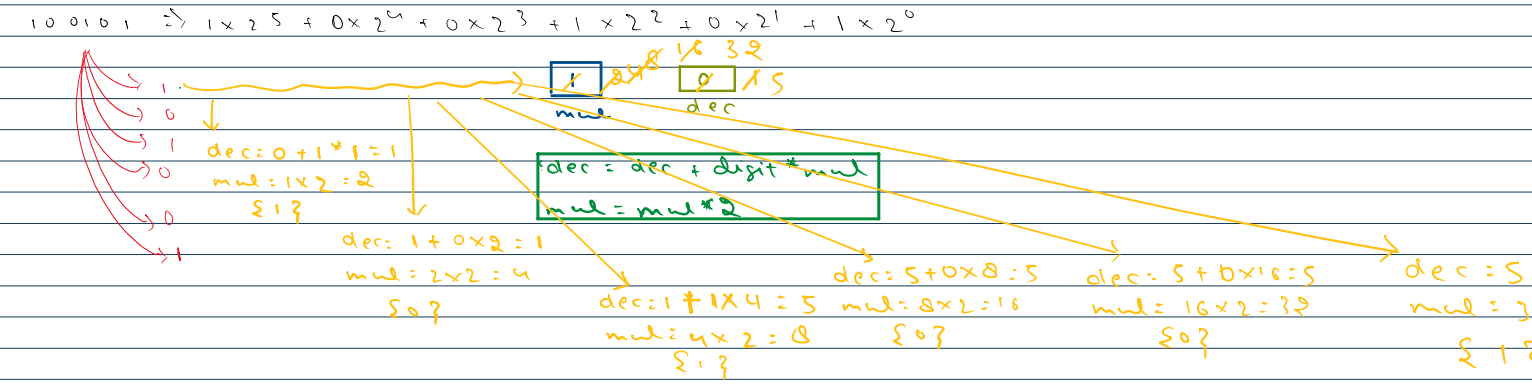
$0 \times 2^{4} + 0 \times 2^{3} + 0 \times 2^{2} + 0 \times 2^{1} + 0 \times 2^{0}$

1×2^{0}

```
        double d = 9.78;
        int i = (int) d;
        System.out.println(i);

        int c = 5;
        double e = (double) a;
        double f = a;
        System.out.println(e);
        System.out.println(f);
    }

}
```

Binary to Decimal

$100101 \Rightarrow 1\times2^5 + 0\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0$




```
public class Main {
    public static void main(String[] args) {
        int n = 100101;
        int dec = 0;
        int mul = 1;

        while(n > 0){
            int digit = n%10;
            dec = dec + digit*mul;
            mul = mul*2;

            n = n/10;
        }
        System.out.println(dec);
    }

}
```

+1×33 = 37

9 × 2 = 64